

דוח מסכם -

פרויקט גמר בקורס רשתות תקשורת מחשבים

חלק 1 - אריזת נתונים ולכידת מנות בעזרת WIRESHARK

דרך יצירת קובץ CSV עם הודעות בשכבת היישום

- בשלב הראשון יצרנו קובץ CSV המכיל הודעות המדמות תקשורת בשכבת היישום.
- הקובץ כולל שדות כגון מזהה הודעה, פרוטוקול יישום, מקור ויעד ההודעה, תוכן ההודעה וחותמת זמן שבה ההודעה נשלחה.
- קובץ ה-CSV נוצר באופן ידני לצורך הפרויקט, במטרה לייצג הודעות טיפוסיות הנשלחות בין לקוח לשרת בשכבת היישום.
- הקובץ שימש כקלט למחברת Jupyter, אשר באמצעותה בוצעה הדמיה של תהליך אריזת הנתונים והפקת תעבורת רשת בפועל.

תיאור והסבר של תהליך אריזה של מנות

	A	B	C	D	E	F
1	msg_id	app_protocol	src_app	dst_app	message	timestamp
2	1	HTTP	browser	web_server	GET / HTTP/1.1\nHost: example.com	0.01
3	2	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: text/html	0.02
4	3	HTTP	browser	web_server	GET /style.css HTTP/1.1\nHost: example.com	0.03
5	4	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: text/css	0.04
6	5	HTTP	browser	web_server	GET /main.js HTTP/1.1\nHost: example.com	0.05
7	6	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: application/javascript	0.06
8	7	HTTP	browser	web_server	GET /logo.png HTTP/1.1\nHost: example.com	0.07
9	8	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: image/png	0.08
10	9	HTTP	browser	web_server	GET /fonts/main.woff2 HTTP/1.1\nHost: example.com	0.09
11	10	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: font/woff2	0.1
12	11	HTTP	browser	web_server	GET /api/user HTTP/1.1\nHost: example.com	0.12
13	12	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: application/json	0.135
14	13	HTTP	browser	web_server	POST /api/login HTTP/1.1\nHost: example.com\nusername=alice	0.16
15	14	HTTP	web_server	browser	HTTP/1.1 200 OK\nSet-Cookie: session=abc123	0.175
16	15	HTTP	browser	web_server	GET /dashboard HTTP/1.1\nHost: example.com	0.2
17	16	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: text/html	0.215
18	17	HTTP	browser	web_server	GET /dashboard.js HTTP/1.1\nHost: example.com	0.23
19	18	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: application/javascript	0.245
20	19	HTTP	browser	web_server	GET /avatar.png HTTP/1.1\nHost: example.com	0.26
21	20	HTTP	web_server	browser	HTTP/1.1 200 OK\nContent-Type: image/png	0.275

תהליך אריזת הנתונים מדמה את פעולת מודל TCP/IP, שבו כל שכבה מוסיפה מידע משלה לנתונים:

- + תחילה, בשכבת היישום, קיימת ההודעה הגולמית כפי שנוצרה בקובץ ה-CSV. בהמשך, שכבת התעבורה (TCP) מוסיפה שדות כגון מספרי פורטים, מספרי רצף ואישורים, על מנת לאפשר תקשורת אמינה בין נקודות הקצה.
- + לאחר מכן, שכבת הרשת (IP) מוסיפה כתובות מקור ויעד, האחריות על ניתוב המידע בין מחשבים שונים ברשת.
- + לבסוף, שכבת הקישור עוטפת את המידע למסגרת המתאימה להעברה פיזית. במחברת Jupyter ניתן לראות הדמיה ברורה של כל שלב בתהליך, המדגישה כיצד אותה הודעה מקבלת מבנה שונה בכל שכבה.

תיאור והסבר של תהליך הלכידה

- + לאחר יצירת התעבורה, הופעלה תוכנת Wireshark על ממשק הרשת המקומי.
- + במהלך הרצת המחברת, נשלחו מנות TCP, אשר נלכדו בזמן אמת על ידי Wireshark.
- + בוצע שימוש במסננים בסיסיים (כגון סינון לפי פורט TCP) על מנת להתמקד רק בתעבורה הרלוונטית לפרויקט.
- + לאחר עצירת הלכידה, התעבורה נשמרה לקובץ בפורמט pcap לצורך ניתוח נוסף. שלב זה נתן דוגמה מוחשית לתעבורת רשת אמיתית הניתנת לצפייה וניתוח.

תיאור והסבר של תעבורה שנלכדה ב- WIRESHARK

- + בלכידה שבוצעה ב- Wireshark (מצורפת תמונה בעמוד הבא), ניתן לראות תקשורת TCP בין הלקוח לשרת, כאשר שני הצדדים פועלים על אותו מחשב באמצעות ממשק ה- Loopback (127.0.0.1).
- + ההודעות נשלחות לפורט השרת ומשתקפות כחבילות TCP המכילות נתונים משכבת היישום.
- + כפי שניתן לראות בצילום המסך, החבילות כוללות דגלי TCP המעידים על שליחת נתונים ואישור קבלתם, וכן מטען נתונים קצר המייצג הודעת טקסט שנשלחה בין רכיבי המערכת.
- + כל חבילה כוללת תוכן טקסטואלי קצר, המייצג הודעה שנשלחה כחלק מהתקשורת בין רכיבי המערכת. הלכידה ממחישה כיצד הודעה פשוטה מועברת בפועל ברשת תוך שימוש במנגנוני האמינות של TCP.

Destination	Protocol	Length	Info
127.0.0.1	TCP	68	57772 → 12345 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2850942364 TSecr=0 SACK_PERM
127.0.0.1	TCP	68	12345 → 57772 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3197512528 TSecr=2850942364 SACK_PERM
127.0.0.1	TCP	56	57772 → 12345 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2850942365 TSecr=3197512528
127.0.0.1	TCP	56	[TCP Window Update] 12345 → 57772 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3197512529 TSecr=2850942365
127.0.0.1	TCP	95	57772 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=39 TSval=2850942365 TSecr=3197512528
127.0.0.1	TCP	56	12345 → 57772 [ACK] Seq=1 Ack=40 Win=408320 Len=0 TSval=3197512529 TSecr=2850942365
127.0.0.1	TCP	118	12345 → 57772 [PSH, ACK] Seq=1 Ack=40 Win=408320 Len=62 TSval=3197512530 TSecr=2850942365
127.0.0.1	TCP	56	57772 → 12345 [ACK] Seq=40 Ack=63 Win=408320 Len=0 TSval=2850942366 TSecr=3197512530

> Frame 51: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface	0000 02 00 00 00 45 00 00 34 00 00 40 00 40 06 00 00	E..4..@...
> Null/Loopback	0010 7f 00 00 01 7f 00 00 01 e1 ac 30 39 2f 82 0e 80	09/...
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020 3c 24 3d 06 80 10 18 ec fe 28 00 00 01 01 08 0a	<\$.-.....(.....
> Transmission Control Protocol, Src Port: 57772, Dst Port: 12345, Seq: 1, Ack: 1, Len: 0	0030 a9 ed ed 9d be 96 2b 50	+P

חלק 2: יצירת יישום וניתוח תעבורה

הסבר כללי על מערכת ומבנה הקוד

- המערכת היא יישום צ'אט מבוסס Client Server מעל TCP.
- השרת מאזין על פורט קבוע, מקבל חיבורים ממספר לקוחות בו זמנית, ומשמש כמתווך להעברת הודעות בין לקוחות.
- קוד המערכת מחולק לשני רכיבים עיקריים:
 - ❖ קוד השרת - אחראי על פתיחת Socket עם האזנה לפורט קבוע וקבלת חיבורים מלקוחות. עבור כל לקוח שמתחבר נוצר תהליך נפרד, המטפל בקבלת הודעות מהלקוח ובהעברתם ללקוחות אחרים דרך השרת. מבנה זה מאפשר טיפול במספר לקוחות במקביל ללא חסימה.
 - ❖ קוד הלקוח - כולל ממשק משתמש גרפי (GUI) המאפשר למשתמש להתחבר לשרת, לשלוח הודעות ולקבל הודעות נכנסות בצורה נוחה. קבלת ההודעות מהשרת מתבצעת בתהליך נפרד מה-GUI כך שממשק המשתמש נשאר תגובתי גם בזמן תקשורת רשת פעילה.
- השרת מנהל מבנה נתונים פנימי של משתמשים מחוברים, הכולל שיוך בין שם משתמש לחיבור הרשת שלו, לצורך ניתוב הודעות והצגת משתמשים פעילים.
- הפרדה זו בין לוגיקת התקשורת לבין ממשק המשתמש תורמת לקריאות הקוד, לתחזוקה פשוטה ולהרחבה עתידית של המערכת.

אסינכרוניות וממשק משתמש גרפי (GUI)

המערכת עושה שימוש באסינכרוניות הן בצד השרת והן בצד הלקוח. בצד השרת, כל לקוח מטופל באמצעות Thread נפרד, מה שמאפשר קבלת חיבורים ממספר לקוחות בו זמנית והעברת הודעות במקביל ללא תלות בקצב הפעולה של לקוח בודד.

בצד הלקוח, יושם ממשק משתמש גרפי (GUI) המאפשר חוויית שימוש נוחה וברורה. קבלת ההודעות מהשרת מתבצעת בתהליך נפרד מהרצת ה-GUI כך שהממשק אינו נחסם בזמן המתנה לנתונים מהשרת. שילוב זה מדגים עקרונות של עבודה מקבילית והפרדת אחריות בין שכבת התקשורת לשכבת הממשק.

```
TCP Chat Client
Name: Yonatan Host: 127.0.0.1 Port: 12345

[connected to 127.0.0.1:12345 as Yonatan]
SYS Hello Yonatan. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Yonatan joined the chat.
FROM SERVER: Yonatan joined the chat.
FROM SERVER: Yosef left the chat.
[disconnected]
SYS Connection closed by server.
SYS Welcome. Please identify: HELLO <your_name>
[connected to 127.0.0.1:12345 as Yonatan]
SYS Hello Yonatan. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Yonatan joined the chat.
FROM SERVER: Yosef joined the chat.
FROM SERVER: Moshe left the chat.
FROM SERVER: Sarah left the chat.
FROM SERVER: Noya left the chat.
[disconnected]
SYS Connection closed by server.
```

Tips: /users | /dm <name> <msg> | /all <msg> (or just type to broadcast)

```
TCP Chat Client
Name: Moshe Host: 127.0.0.1 Port: 12345

SYS Welcome. Please identify: HELLO <your_name>
[connected to 127.0.0.1:12345 as Moshe]
SYS Hello Moshe. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Moshe joined the chat.
FROM SERVER: Sarah joined the chat.
FROM SERVER: Noya joined the chat.
FROM SERVER: Yonatan joined the chat.
FROM SERVER: Yosef joined the chat.
[disconnected]
SYS Connection closed by server.
```

Tips: /users | /dm <name> <msg> | /all <msg> (or just type to broadcast)

```
TCP Chat Client
Name: Sarah Host: 127.0.0.1 Port: 12345 Connect Disconnect

SYS Welcome. Please identify: HELLO <your_name>
[connected to 127.0.0.1:12345 as Sarah]
SYS Hello Sarah. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Sarah joined the chat.
[disconnected]
SYS Connection closed by server.
SYS Welcome. Please identify: HELLO <your_name>
[connected to 127.0.0.1:12345 as Sarah]
SYS Hello Sarah. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Sarah joined the chat.
FROM SERVER: Noya joined the chat.
FROM SERVER: Yonatan joined the chat.
FROM SERVER: Yosef joined the chat.
FROM SERVER: Moshe left the chat.
[disconnected]
SYS Connection closed by server.
```

```
TCP Chat Client
Name: Noya Host: 127.0.0.1 Port: 12345 Connect

FROM SERVER: Noya joined the chat.
ME: /users
SYS Users: Noya, Yonatan, Yosef
FROM SERVER: Yosef left the chat.
FROM SERVER: Yonatan left the chat.
SYS Connection closed by server.
[disconnected]
SYS Welcome. Please identify: HELLO <your_name>
[connected to 127.0.0.1:12345 as Noya]
SYS Hello Noya. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Noya joined the chat.
FROM SERVER: Yonatan joined the chat.
FROM SERVER: Yosef joined the chat.
FROM SERVER: Moshe left the chat.
FROM SERVER: Sarah left the chat.
[disconnected]
SYS Connection closed by server.
```

```
TCP Chat Client
Name: Yosef Host: 127.0.0.1 Port: 12345 Connect

[connected to 127.0.0.1:12345 as Yosef]
SYS Hello Yosef. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Yosef joined the chat.
FROM SERVER: Yonatan joined the chat.
FROM SERVER: Noya joined the chat.
[disconnected]
SYS Connection closed by server.
SYS Welcome. Please identify: HELLO <your_name>
[connected to 127.0.0.1:12345 as Yosef]
SYS Hello Yosef. Commands: /users, /dm <name> <msg>, /all <msg>
FROM SERVER: Yosef joined the chat.
FROM SERVER: Moshe left the chat.
FROM SERVER: Sarah left the chat.
```

הוראות התקנה והרצה

דרישות מקדימות -

- ❖ התקנת Python 3 במערכת
- ❖ אין צורך בספריות חיצוניות
- ❖ התקנת Wireshark לצורך ניתוח תעבורת הרשת

הרצת השרת -

- (1) פתח חלון טרמינל
- (2) נווט לתיקיית הפרויקט
- (3) הרץ את השרת באמצעות הפקודה הבאה:
`python3 server.py`

לאחר ההרצה, השרת יאזין לחיבורים נכנסים בפרוטוקול TCP על פורט 12345.

הרצת הלקוחות -

לצורך בדיקת המערכת יש להריץ מספר לקוחות במקביל באמצעות ממשק המשתמש הגרפי.

עבור כל לקוח:

- ❖ פתח חלון טרמינל חדש
- ❖ נווט לתיקיית הפרויקט
- ❖ הרץ את הלקוח באמצעות הפקודה:
`python3 client_gui.py`

בחלון ה-GUI יש להזין שם משתמש ייחודי ולבצע התחברות לשרת.

פקודות בצד בלקוח -

- ❖ `/users` - הצגת רשימת משתמשים מחוברים
- ❖ `/dm <username> <message>` - שליחת הודעה פרטית למשתמש אחר
- ❖ הקלדת טקסט חופשי - שליחת הודעה בצ'אט הפעיל לכל המשתמשים המחוברים

הערות -

במידה והשרת והלקוחות מורצים על אותו מחשב, יש לבצע את ניתוח תעבורת הרשת ב-Wireshark על ממשק ה-Loopback.

דוגמאות קלט ופלט

בתמונה ניתן לראות מספר מופעים של השיח בין המשתמשים בצ'אט, המחוברים במקביל לשרת.

כל חלון מייצג משתמש שונה במערכת (Moshe, Yonatan, Noya, Yosef, Sarah)

כאשר כולם מחוברים לאותו שרת.

השרת מנהל את רשימת המשתמשים המחוברים ומנתב את ההודעות בהתאם לפעולות המשתמשים.

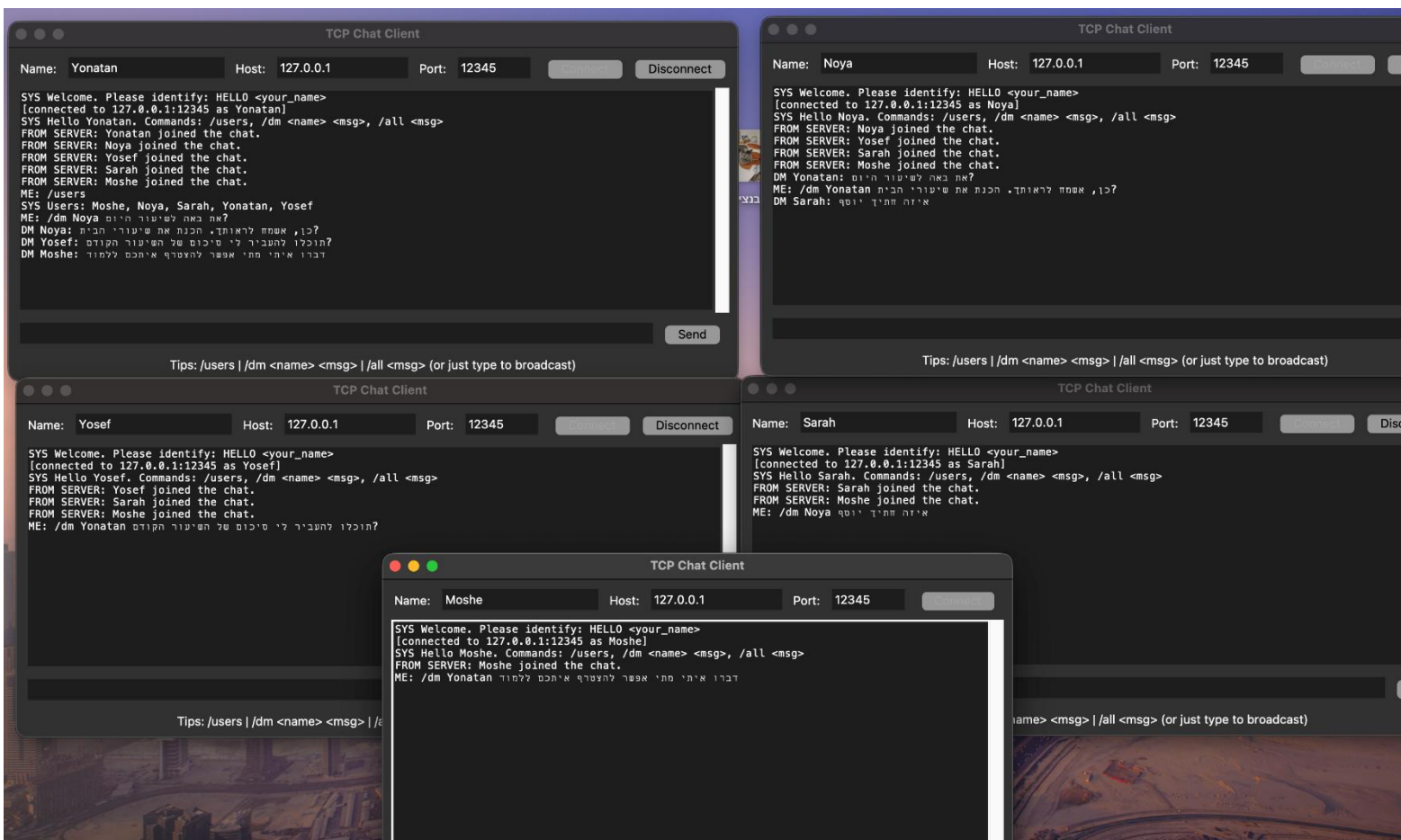
האינטראקציה מדגימה שליחת הודעות פרטיות בין משתמשים באמצעות הפקודה /dm, כאשר ההודעות מועברות מהלקוח לשירות, וממנו ללקוח היעד בלבד.

ניתן לראות שכל משתמש מקבל רק את ההודעות שיועדו אליו, בעוד שמשתמשים אחרים אינם חשופים לתוכן ההודעה.

המערכת תומכת במספר חיבורים פעילים בו זמנית, כאשר כל משתמש יכול לשלוח ולקבל הודעות ללא תלות בקצב הפעולה של משתמשים אחרים.

התמונה ממחישה את פעולתה התקינה של מערכת הצ'אט, את ניהול החיבורים על ידי השרת, ואת השילוב בין ממשק משתמש גרפי נוח לבין תקשורת רשת אסינכרונית ויציבה.

בקוד הוגדר timeout גבוה שלא מנתק את הצ'אט למשתמשים, כל עוד יש פעילות.



ניתוח תעבורה של היישום עד שכבת הרשת

בשלב זה בוצע ניתוח של תעבורת הרשת שנוצרה על ידי יישום הצ'אט, במטרה להבין כיצד הודעות היישום מועברות בפועל ברשת, החל משכבת היישום ועד שכבת הרשת. לשם כך נעשה שימוש בכלי Wireshark, אשר אפשר צפייה בחבילות הרשת בזמן אמת.

הליכידה בוצעה על ממשק ה- Loopback, מאחר והשרת והלקוחות הורצו על אותו מחשב.

יצירת חיבור בין לקוח לשרת -

עם הפעלת הלקוח, מתבצע תהליך פתיחת חיבור TCP מול השרת. תהליך זה נראה ב-Wireshark כרצף של שלוש חבילות עוקבות, המוכרות כ- Three Way Handshake:

(1) בקשת חיבור מהלקוח לשרת.

(2) אישור מהשרת.

(3) אישור סופי מהלקוח.

שלב זה מתרחש לפני כל העברת נתונים, ומטרתו להבטיח חיבור אמין בין הצדדים.

העברת הודעות היישום -

לאחר יצירת החיבור, כל פעולה של המשתמש ביישום הצ'אט (כגון התחברות, פתיחת צ'אט או שליחת הודעה) יוצרת תעבורת רשת חדשה. הודעות אלו נשלחות מהלקוח לשרת, ומועברות על ידי השרת ללקוח היעד.

ב-Wireshark ניתן לראות כי הודעות היישום עצמן אינן נשלחות כחבילות נפרדות, אלא משולבות בתוך חבילות TCP. תוכן ההודעה מופיע כחלק מה- Payload של החבילה, בעוד שמנגנוני TCP אחראים על אמינות ההעברה.

אמינות וסדר ההודעות -

פרוטוקול TCP דואג לכך שכל הודעה תגיע ליעדה בשלמות ובסדר הנכון. הדבר בא לידי ביטוי במספרי רצף ואישורים (ACK) שניתן לראות בפרטי החבילות. מנגנונים אלו פועלים באופן שקוף ליישום עצמו, אך מבטיחים שהודעות הצ'אט יגיעו ללא אובדן או כפילות.

שכבת הרשת (IP) -

בניתוח החבילות ניתן לראות גם את פרטי שכבת הרשת, הכוללים כתובת מקור וכתובת יעד. במקרה זה, מאחר והיישום הופעל על אותו מחשב, כתובות ה-IP הן כתובות Loopback. במצב של עבודה בין מחשבים שונים, כתובות אלו היו מוחלפות בכתובות רשת מקומית.

סיום חיבור -

בעת יציאה של לקוח מהיישום או סיום צ'אט, מתבצע תהליך סגירת חיבור TCP. תהליך זה מתבטא בחבילות סיום, אשר מסמנות לשני הצדדים כי החיבור הסתיים בצורה מסודרת.

```
> Frame 49: Packet, 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 57772, Dst Port: 12345, Seq: 0, Len: 0
```

Destination	Protocol	Length	Info
127.0.0.1	TCP	68	57772 → 12345 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2850942364 TSecr=0 SACK_PERM
127.0.0.1	TCP	68	12345 → 57772 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3197512528 TSecr=2850942364 SACK_PERM
127.0.0.1	TCP	56	57772 → 12345 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2850942365 TSecr=3197512528
127.0.0.1	TCP	56	[TCP Window Update] 12345 → 57772 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3197512529 TSecr=2850942365
127.0.0.1	TCP	95	57772 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=39 TSval=2850942365 TSecr=3197512528
127.0.0.1	TCP	56	12345 → 57772 [ACK] Seq=1 Ack=40 Win=408320 Len=0 TSval=3197512529 TSecr=2850942365
127.0.0.1	TCP	118	12345 → 57772 [PSH, ACK] Seq=1 Ack=40 Win=408320 Len=62 TSval=3197512530 TSecr=2850942365
127.0.0.1	TCP	56	57772 → 12345 [ACK] Seq=40 Ack=63 Win=408320 Len=0 TSval=2850942366 TSecr=3197512530

סעיף בונוס - פיתוח מתקדם בצד שרת ולקוח

בפרויקט יושמו עקרונות פיתוח מתקדמים, כולל עבודה אסינכרונית וממשק משתמש גרפי.

השרת תומך במספר לקוחות במקביל באמצעות ריבוי תהליכים, והלקוח עושה שימוש ב-GUI המופרד מלוגיקת התקשורת. שילוב זה משפר את ביצועי המערכת, את חוויית המשתמש ואת יציבות התקשורת, ועומד בקריטריונים לפיתוח מתקדם כפי שהוגדרו בפרויקט.

חלק 3: תיאור שימוש בבינה מלאכותית

מטרות השימוש

במהלך ביצוע הפרויקט נעשה שימוש בכלי בינה מלאכותית כעזר תומך בתהליך הלמידה, הפיתוח והתיעוד. מטרת השימוש לא הייתה להחליף את הבנת החומר או את עבודת הסטודנטים, אלא לשמש ככלי מסייע לקבלת הסברים, חידוד רעיונות ושיפור איכות המימוש והדוח.

השימוש בבינה מלאכותית כלל את המטרות הבאות:

- ❖ סיוע בתכנון מבנה כללי של מערכת Client Server מבוססת TCP.
- ❖ קבלת דוגמאות לקוד בסיסי לתקשורת באמצעות Sockets.
- ❖ שיפור קריאות הקוד והוספת תיעוד והערות.
- ❖ סיוע בניסוח הסברים לדוח, בדגש על ניתוח תעבורה והקשר בין שכבות המודל.
- ❖ קבלת הכוונה בשימוש בכלי Wireshark לצורך ניתוח התעבורה.
- ❖ חשוב להדגיש כי כל הקוד נבדק, הובן והותאם לדרישות הפרויקט, והשימוש בבינה מלאכותית שימש ככלי עזר בלבד.

דוגמה 1 - תכנון ארכיטקטורת המערכת:

- ❖ קלט (Prompt): בקשה להסבר כיצד ניתן לממש מערכת צ'אט מבוססת שרת ולקוחות מעל TCP, כולל ריבוי לקוחות בו זמנית.
- ❖ פלט (תוצר): הצעה למבנה כללי הכולל שרת מאזין על פורט קבוע, שימוש ב- Thread לכל לקוח, וניהול רשימת משתמשים מחוברים לצורך ניתוב הודעות בין לקוחות.

דוגמה 2 - סיוע בכתיבת קוד:

- ❖ קלט (Prompt): בקשה לדוגמה לקוד Python המממש תקשורת Client Server באמצעות TCP Sockets, כולל שליחת וקבלת הודעות.
- ❖ פלט (תוצר): קוד בסיסי לשרת ולקוח, אשר שימש כשלד ראשוני. הקוד הותאם והורחב לצורך מימוש דרישות הפרויקט, כגון זיהוי משתמשים, פתיחת צ'אט בין לקוחות וטיפול בניתוקים.

דוגמה 3 - שיפור תיעוד וקריאות הקוד:

- ❖ קלט (Prompt): בקשה להוספת הערות והסברים באנגלית לקוד קיים.
- ❖ פלט (תוצר): הוספת הערות המסבירות את תפקיד כל חלק בקוד, זרימת התקשורת בין הלקוח לשרת, והקשר בין הקוד לבין עקרונות תקשורת רשת.

דוגמה 4 - ניתוח תעבורה:

- ❖ קלט (Prompt): בקשה להסבר כיצד לנתח תעבורת TCP של יישום צ'אט באמצעות Wireshark.
- ❖ פלט (תוצר): הכוונה לבחירת ממשק לכידה מתאים, שימוש בפילטרים רלוונטיים, וזיהוי של שלבי פתיחת חיבור, העברת הודעות וסיום חיבור.

דוגמה 5 - מימוש מערכת צ'אט מבוססת שרת ולקוחות:

- ❖ קלט (Prompt): בקשה להסבר כיצד ניתן לממש מערכת צ'אט מבוססת שרת ולקוחות מעל TCP, כולל ריבוי לקוחות בו זמנית.
- ❖ פלט (תוצר): הצעה למבנה כללי הכולל שרת מאזין על פורט קבוע, שימוש ב- Thread לכל לקוח, וניהול רשימת משתמשים מחוברים לצורך ניתוב הודעות בין לקוחות.